

Rockchip 多目拼接模型标定指南

发布版本：V1.2.0

日期：2024.4.25

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2023 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

本文为应用多目拼接方案进行开发的工程师而写，目的是介绍多目拼接模型标定基本原理、操作步骤及使用注意事项等内容。

产品版本

与本文档相对应的产品版本如下：

产品名称	产品版本
RV1103b	
RV1106b	
RV1106	
RV1126	
RK3588	
RK3576	

修订记录

日期	版本	作者	修改说明
2023.06.20	V1.1.1	mingpei.liu	初始版本
2023.07.10	V1.1.2	mingpei.liu	完善拍图注意事项
2023.11.29	V1.1.3	mingpei.liu	完善拍图注意事项
2024.03.19	V1.1.4	zhipeng.ye	增加板端采集标定图流程
2024.04.25	V1.2.0	mingpei.liu	完善文档说明

1. 概述

1.1 多目拼接概述

多目拼接可以将多路输入图像拼接为一路图像，从而实现获取更大视野的需求。

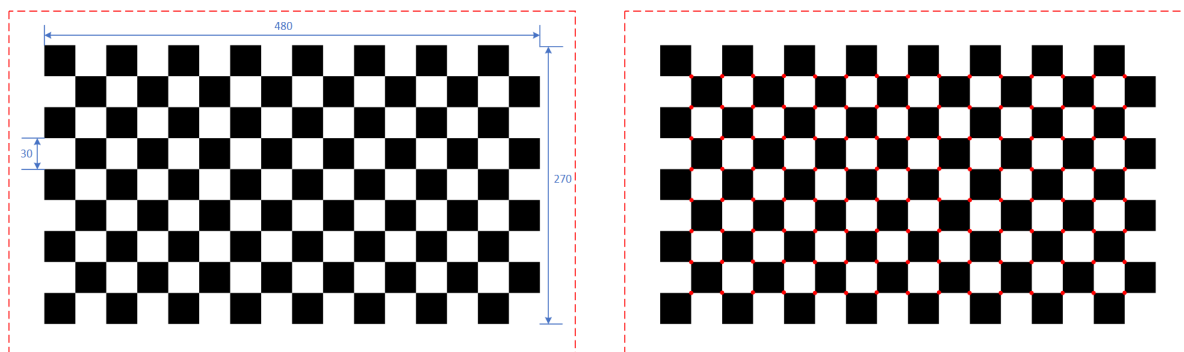
1.2 模型标定概述

对于结构固定的产品，模型标定只需要进行一次，可以在同一批次产品中选择一个结构最佳的机器进行模型标定。标定时需要拍摄大量棋盘格图像，然后经过AVS模型标定工具生成标定文件。标定参数包括镜头的各自的内参、相邻相机的相对外参。内参包括焦距（ f_x, f_y ），主点位置（ u_0, v_0 ）以及光学失真系数（ k_1, k_2, k_3, p_1, p_2 ）；相对外参包括三轴角度 $Tilt, Pan, Roll$ ， x 轴 y 轴 z 轴的平移参数；同时返回所得标定参数的相对误差。

2. 模型标定

2.1 棋盘格规格设计

图2-1棋盘格规格设计



如图2-1为模组标定中使用的棋盘格，是对称的方形棋盘格，棋盘格的具体参数如下：

- 1) 棋盘格的长为480mm，宽为270mm；
- 2) 棋盘格中每个方块的物理尺寸是30mm，精度要求为0.1mm；
- 3) 棋盘格要求表面平整；
- 4) 棋盘格的长边共16个方格，短边共9个方格；
- 5) 图中红色圈所标出的为内点，共15*8个内点。

注意：棋盘格四周须有留白区域，如图2-1所示，棋盘格四周与红色边框之间有留白区域，留白区域的边长大于一个棋盘格即可。

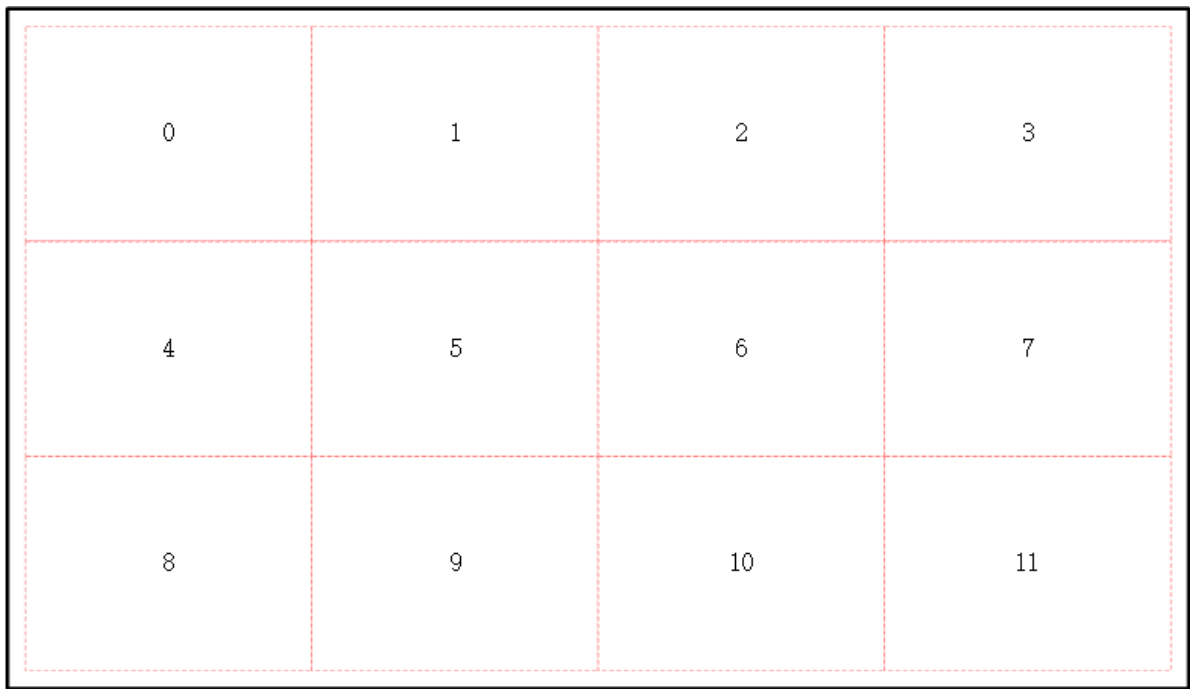
以上为设计案例，实际使用中尺寸及棋盘格个数可自由设计。

2.2 内参拍图

该场景用于标定单个相机的内参（焦距、光心和畸变系数），具体拍图要求如下：

- 1) 每个镜头至少拍取 10 张图像。保存时以<固定前缀名><镜头号>_<帧序号><格式后缀> 来对图像进行命名；
- 2) 拍图时，将棋盘格置于镜头前方，不断变换棋盘格方向、距离，改变其在视野中的位置，尽可能的使棋盘格覆盖视野中的所有位置；
- 3) 对于非鱼眼镜头，近距离棋盘格约占画面 1/4 即可，远距离约占画面 1/8；对于鱼眼镜头，由于 FOV 较大，近距离棋盘格约占画面 1/5，远距离约占画面 1/10 左右，抓拍时建议将棋盘格均匀绕鱼眼镜头一周，采集不同位置的棋盘格图像。

图2-2 内参标定棋盘格拍摄分布示意图



如图2-2所示，为内参拍摄的示意图，拍摄时尽可能的让棋盘格覆盖相机视野。实际拍摄效果图如图2-3所示。

图2-3 内参标定实拍图



拍图命名规则为：“camera”+相机编号+“_”+图像序号（3位数，不足3位数的前面补0），例如0号相机第1幅图为“camera0_000”，0号相机第2幅图为“camera0_001”，2号相机第0幅图为“camera2_000”，2号相机第10幅图为“camera2_009”，保存格式为“.jpg", ".yuv", ".bin"等常用格式。

拍图注意事项

- 1) 拍摄棋盘格图片必须清晰，不能模糊；
- 2) 整个拍摄过程中，相机需保持定焦，不能随意调焦，一旦调节，整个标定过程需重新进行；
- 3) 标定环境需光照充足且均匀；
- 4) 拍摄过程中，视野内只能有一个棋盘格；
- 5) 拍摄过程中棋盘格不能存在反光现象；
- 6) 拍摄到的棋盘格需完整，不能存在缺失现象；
- 7) 拍摄的棋盘格不能存在过曝或者过暗的情况；

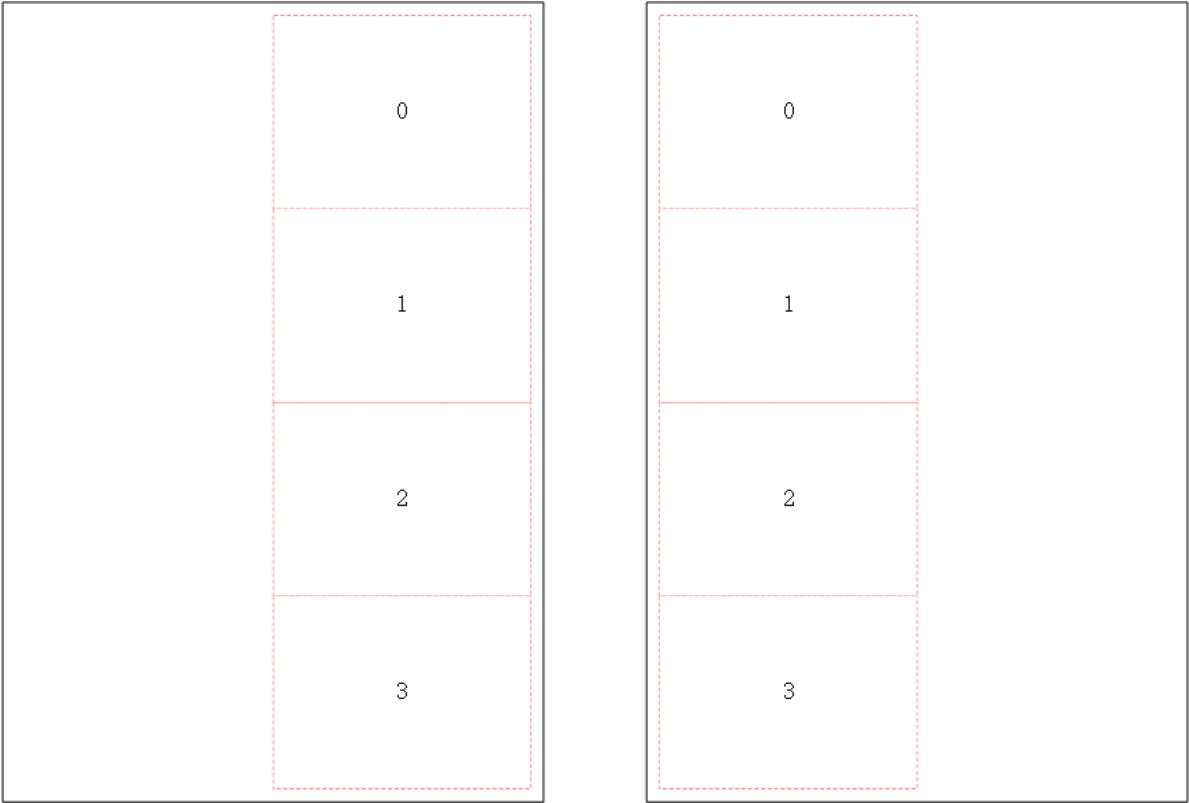
8) 棋盘格需覆盖不同角度、不同区域和不同距离。

2.3 外参拍图

该场景用于标定相邻两个相机间的外参（旋转量和平移量），具体拍图要求如下：

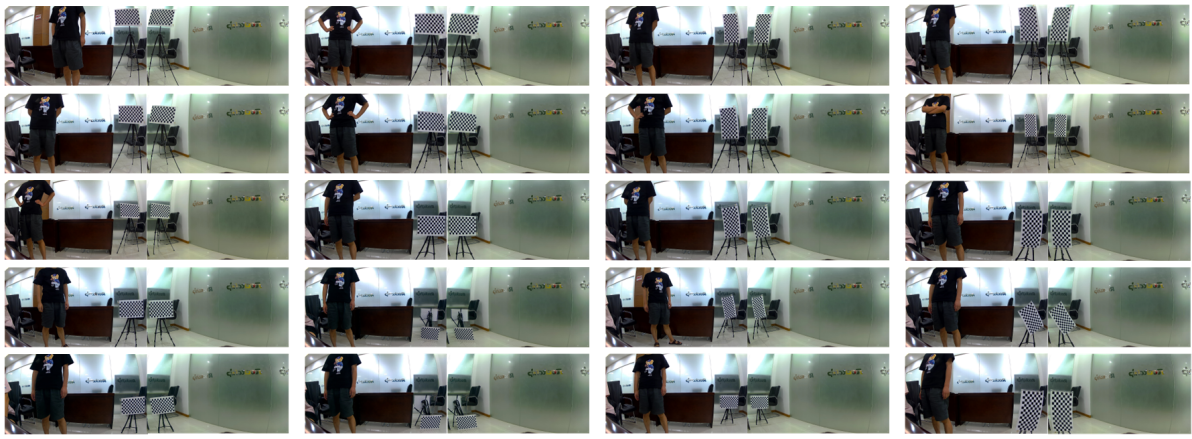
- 1) 每组镜头（相邻两个镜头）至少抓拍 10 对图片。保存时以<固定前缀名><镜头号>_<较小镜头号><较大镜头号>_<帧序号><格式后缀> 来对图像进行命名。
- 2) 拍图时，首选确定是哪两个相机拍摄，保证两个相机同时拍摄同一个棋盘格。棋盘格的摆放位置应全部位于两个相机的重叠区域范围内，并在该范围移动，尽可能地覆盖全部重叠区域，拍摄过程中应对棋盘格做旋转和远近距离的改变，保证最少涵盖三种拍摄距离。
- 3) 在移动棋盘格的过程中应保证棋盘格能够清晰拍到。
- 4) 拍摄过程中需要保证两个相机同步出图，模组静止，不可有震动、晃动等干扰模组状态的情况出现。

图2-4外参标定棋盘格拍摄分布示意图



如图2-4所示，拍摄外参标定所需的棋盘格图像时，需保证左右两个相机同时拍摄到完整的棋盘格。实际的拍摄效果如图3-5所示。

图2-5 外参标定实拍图



拍图命名规则为：“camera”+相机编号+“_”+两两相机的编号+“_”+图像序号（3位数，不足3位数的前面补0），例如0号相机与1号相机拍摄到的第5幅图为：“camera0_01_004”和“camera1_01_004”，2号相机与3号相机拍摄到的第11幅图为：“camera2_23_010”和“camera3_23_010”，3号相机与0号相机拍摄到的第1幅图为：“camera3_30_000”和“camera0_30_000”，保存格式为“.jpg",".yuv",".bin"等常用格式。

拍图注意事项

- 1) 拍摄棋盘格图片必须清晰，不能模糊；
- 2) 整个拍摄过程中，相机需保持定焦，不能随意调焦，一旦调节，整个标定过程需重新进行；
- 3) 标定环境需光照充足且均匀；
- 4) 拍摄过程中，视野内只能有一个棋盘格；
- 5) 拍摄过程中棋盘格不能存在反光现象；
- 6) 拍摄到的棋盘格需完整，不能存在缺失现象；
- 7) 拍摄的棋盘格不能存在过曝或者过暗的情况；
- 8) 拍摄过程中需要保证两个相机同步出图，模组保持静止；
- 9) 棋盘格需覆盖不同角度、不同区域和不同距离。

2.4 内外参标定

将拍摄好的棋盘格图像放入指定文件夹，通过AVS标定工具进行标定，获取标定后的文件。

3. API接口及Demo

3.1 接口定义

```
/*
 * @enum      rkAVS_CALIB_STATUS_E
 * @brief     the return status of function
 */
typedef enum rkAVS_CALIB_STATUS_E
{
    RK_AVS_CALIB_STATUS_EOF = -1,                /**< error of function inside */
}
```

```

    RK_AVS_CALIB_STATUS_OK = 0,                /**< run successfully */
    RK_AVS_CALIB_STATUS_FILE_READ_ERROR,       /**< error: fail to read file */
    RK_AVS_CALIB_STATUS_FILE_WRITE_ERROR,      /**< error: fail to write file */
    RK_AVS_CALIB_STATUS_INVALID_PARAM,         /**< error: invalid parameter */
    RK_AVS_CALIB_STATUS_ALLOC_FAILED,          /**< error: fail to alloc buffer */
    RK_AVS_CALIB_STATUS_BUTT                    /**< reserved fields */
}AVS_CALIB_STATUS_E;

```

rkAVS_CALIB_STATUS_E定义了算法库内部的状态信息。

```

/*
 * @enum      rkAVS_CALIB_IMAGE_FORMAT_E
 * @brief     Specify the image format.
 * @note     Have gray, rgb, yuvnv12.
 */
typedef enum rkAVS_CALIB_IMAGE_FORMAT_E {
    RK_AVS_IMAGE_FORMAT_TYPE_GRAY = 0,          /**<gray format*/
    RK_AVS_IMAGE_FORMAT_TYPE_RGB = 1,           /**<rgb format*/
    RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12 = 2,       /**<yuvnv12
format*/
} AVS_CALIB_IMAGE_FORMAT_E;

```

rkAVS_CALIB_IMAGE_FORMAT_E定义了输入图像的基本格式，包括灰度图、RGB图和YUVNV12图。

```

/*
 * @enum      rkAVS_CALIB_IMAGE_PARAMS_S
 * @brief     calibration image params.
 */
typedef struct rkAVS_CALIB_IMAGE_PARAMS_S
{
    uint32_t u32ImageWidth;                      /**<image
width*/
    uint32_t u32ImageHeight;                     /**<image
height*/
    uint32_t u32ImageStride;                     /**<image
stride*/
    AVS_CALIB_IMAGE_FORMAT_E eImageFormat;       /**<image format*/
}AVS_CALIB_IMAGE_PARAMS_S;

```

rkAVS_CALIB_IMAGE_PARAMS_S定义了输入图像的基本信息，保留图像的宽、高、stride和图像格式。

```

/*
 * @enum      rkAVS_CALIB_BOARD_PARAMS_S
 * @brief     chess board params.
 */
typedef struct rkAVS_CALIB_BOARD_PARAMS_S
{
    uint32_t u32BoardWidth;
    /**<coners number of board width*/
    uint32_t u32BoardHeight;                      /**
<coners number of board height*/
    uint32_t u32SquareSize;
    /**<square size*/
}AVS_CALIB_BOARD_PARAMS_S;

```



```

    const char* pcImageNameFormat;
    <image name format .yuv, .png, .jpg, .bmp*/
}AVS_CALIB_MODEL_CONFIG_S;

```

AVS_CALIB_MODEL_CONFIG_S为模型标定的输入参数，包括图像信息、棋盘格信息、相机信息、是否保存角点的Flag、输入图像所在文件夹路径、最终标定结果的输出路径以及输入图像的后缀名。

```

/*
 * @enum    rkAVS_STEREO_CALIB_POSE_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_POSE_S
{
    float f32RotationAngle[3];
    float f32Translation[3];
}AVS_STEREO_CALIB_POSE_S;

```

rkAVS_STEREO_CALIB_POSE_S定义了模组的位姿信息，包括旋转角度和位移量。

```

/*
 * @enum    rkAVS_STEREO_CALIB_RMS_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_RMS_S
{
    float f32LeftInternalRms;
    float f32RightInternalRms;
    float f32StereoExternalRms;
}AVS_STEREO_CALIB_RMS_S;

```

rkAVS_STEREO_CALIB_RMS_S定义了标定的误差，包括内参标定误差和外参标定误差，该误差值越小越好，大于0.5效果较差。

```

/*
 * @enum    rkAVS_STEREO_CALIB_CHECK_S
 * @brief   output params.
 */
typedef struct rkAVS_STEREO_CALIB_CHECK_S
{
    AVS_STEREO_CALIB_POSE_S stPose;
    AVS_STEREO_CALIB_RMS_S stRms;
}AVS_STEREO_CALIB_CHECK_S;

```

rkAVS_STEREO_CALIB_CHECK_S作为标定的输出参数，用于判别标定效果的好坏，模型标定返回的为两个相机间的最大重投影误差和相应姿态。

```

/*
 * @enum   rkAVS_CALIB_FIND_POINTS_S
 * @brief   input params.
 */
typedef struct rkAVS_CALIB_FIND_POINTS_S
{
    AVS_CALIB_IMAGE_DATA_S stImageData;
    /**<image params*/
    AVS_CALIB_BOARD_PARAMS_S stBoardParams;
    /**<board params*/
    const char* pcPointsImagePath;           /**
<points image save path*/
}AVS_CALIB_FIND_POINTS_S;

```

rkAVS_CALIB_FIND_POINTS_S作为角点提取检测的输入参数，用于模型标定时验证拍摄的图像是否可以准确的检测到棋盘格角点。

```
int32_t rkAVS_getCalibVersion(char avsToolVersion[128]);
```

rkAVS_getCalibVersion函数用于获取算法库的版本信息。

```
int32_t rkAVS_findPoints(AVS_CALIB_FIND_POINTS_S* psInputParams);
```

rkAVS_findPoints函数用于获取输入图像的角点图像。

```
int32_t rkAVS_calibModel(AVS_CALIB_MODEL_CONFIG_S* psInputParams,
    AVS_STEREO_CALIB_CHECK_S* stStereoCheckParams);
```

rkAVS_calibModel函数用于模型标定。

3.2 标定Demo

```

srcPath: ./calibModel/all_calib_images/
dstPath: ./calibModel/rk_calib_result/
imageNameFormat: .yuv
    // .jpg、.png、.bmp、.bin、.yuv
cameraNum: 2                                //camera number (2-
8)
cameraType: 1                                //0-pinhole、1-omn、
2-fisheye
cameraFov: 120                               //horizontal fov
boardWidth: 15                               //number of
chessboard corners in horizontal direction
boardHeight: 8                               //number of
chessboard corners in vertical direction
squareSize: 30                               //size of
chessboard(mm)
imageWidth: 1920                             //image width
imageHeight: 1080                            //image height
imageStride: 1920                            //image stride
imageFormat: 2                               //0-gray、1-rgb、2-
yuv

```

```

isSaveCornersImage: 0 //draw corners and
save
useAcurateMode: 0 //Optimize by
Stitch

```

以上为模型标定时算法库的参考配置参数。

```

//保存产线标定筛选参数
int saveCheckParams(std::string saveThreshPath, AVS_STEREO_CALIB_CHECK_S*
stereoCalibCheck)
{
    std::ofstream fout(saveThreshPath);
    if (!fout)
    {
        std::cout << "Open save_thresh_path fail!" << std::endl;
        return 0;
    }
    else
    {
        fout << "leftInternalRms: " << stereoCalibCheck->stRms.f32LeftInternalRms
<< std::endl;
        fout << "rightInternalRms: " << stereoCalibCheck-
>stRms.f32RightInternalRms << std::endl;
        fout << "externalRms: " << stereoCalibCheck->stRms.f32StereoExternalRms
<< std::endl;
        fout << "euler_angles_x: " << stereoCalibCheck-
>stPose.f32RotationAngle[0] << std::endl;
        fout << "euler_angles_y: " << stereoCalibCheck-
>stPose.f32RotationAngle[1] << std::endl;
        fout << "euler_angles_z: " << stereoCalibCheck-
>stPose.f32RotationAngle[2] << std::endl;
        fout << "t_x: " << stereoCalibCheck->stPose.f32Translation[0] <<
std::endl;
        fout << "t_y: " << stereoCalibCheck->stPose.f32Translation[1] <<
std::endl;
        fout << "t_z: " << stereoCalibCheck->stPose.f32Translation[2] <<
std::endl;
        //fout << "is_calib_succeed: " << is_succ << std::endl;
    }
    fout.close();
    return 1;
}

```

saveCheckParams用于保存产线标定的误差及模组信息，该参数可作为标定是否符合预期的判别参数。

```

bool readBinParams(std::string image_path, char* output_params, uint64_t length)
{
    std::ifstream fin;
    //读取raw图
    fin.open(image_path, std::ios::binary);
    if (!fin) {
        std::cerr << "open failed: " << image_path << std::endl;
        return false;
    }
    // read函数读取（拷贝）流中的length各字节到buffer
    fin.read((char*)output_params, length);
}

```



```

        fin.close();
        return true;
    }

```

readBinParams函数用于读取YUV数据。

```

//模型标定读取配置文件
int readCalibModelConfig(std::string readPath, std::vector<std::string>&
stringData, std::vector<int32_t>& int32Data)
{
    std::string stringLine;
    std::ifstream ifs(readPath, std::ifstream::in);
    if (ifs.fail())
    {
        std::cout << "read calib model config fail!" << std::endl;
    }
    int32_t lineNum = 0;
    int32_t valueData = 0;
    while (getline(ifs, stringLine))
    {
        std::stringstream ss(stringLine);
        std::string stringUnit;
        std::getline(ss, stringUnit, ' ');
        std::getline(ss, stringUnit, ' ');
        if (lineNum < 3)
        {
            stringData.push_back(stringUnit);
        }
        else
        {
            valueData = atoi(stringUnit.c_str());
            int32Data.push_back(valueData);
        }
        lineNum++;
    }
    ifs.close();
    return 0;
}

```

readCalibModelConfig函数用于读取模型标定的配置信息。

```

//模型标定Demo
int calibModelDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readCalibModelConfig(configPath, stringData, int32Data);
    std::string input_calib_image_path = stringData[0]; //标定图像所在路径
    std::string output_calib_result_path = stringData[1] +
"rk_2_camera_result.xml";
    std::string input_image_format = stringData[2]; //标定图像后缀名
    //获取版本信息
    char avsCalibVersion[128];
    rkAVS_getCalibVersion(avsCalibVersion);
    //初始化参数
    AVS_CALIB_MODEL_CONFIG_S stInputParams;

```

```

//相机规格
stInputParams.stCameraParams.u32CameraFov = int32Data[2];
stInputParams.stCameraParams.u32CameraNum = int32Data[0];
switch (int32Data[1])
{
case 0:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_PINHOLE;
    break;
case 1:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_OMN;
    break;
case 2:
    stInputParams.stCameraParams.eCameraType = AVS_CAMERA_TYPE_FISHEYE;
    break;
default:
    std::cout << "RK_AVS_CALIB: CameraType error!" << std::endl;
    return 1;
}

//棋盘格规格
stInputParams.stBoardParams.u32BoardHeight = int32Data[4];
stInputParams.stBoardParams.u32BoardWidth = int32Data[3];
stInputParams.stBoardParams.u32SquareSize = int32Data[5];

//图像规格
stInputParams.pcImageNameFormat = input_image_format.c_str();
switch (int32Data[9])
{
case 0:
    stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
    break;
case 1:
    stInputParams.stImageParams.eImageFormat = RK_AVS_IMAGE_FORMAT_TYPE_RGB;
    break;
case 2:
    stInputParams.stImageParams.eImageFormat =
RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
    break;
default:
    std::cout << "RK_AVS_CALIB: ImageFormate error!" << std::endl;
    return 1;
}

stInputParams.stImageParams.u32ImageHeight = int32Data[7];
stInputParams.stImageParams.u32ImageWidth = int32Data[6];
if (stInputParams.stImageParams.eImageFormat == RK_AVS_IMAGE_FORMAT_TYPE_RGB)
    stInputParams.stImageParams.u32ImageStride =
stInputParams.stImageParams.u32ImageWidth * 3;
else
    stInputParams.stImageParams.u32ImageStride = int32Data[8];

//输入输出路径
stInputParams.pcInputPath = input_calib_image_path.c_str();
stInputParams.pcOutputPath = output_calib_result_path.c_str();

//功能开关
stInputParams.u32IsSaveCornersImage = int32Data[10];
stInputParams.u32UseAccurateMode = int32Data[11];

//模型标定
AVS_STEREO_CALIB_CHECK_S stStereoCheckParams;
rkAVS_calibModel(&stInputParams, &stStereoCheckParams);

//保存产线筛选参数
std::string saveCheckPath = stringData[1] + "productCheckParams.txt";

```

```

    saveCheckParams(saveCheckPath, &stStereoCheckParams);
    return 0;
}

```

calibModelDem函数为模型标定调用算法库完成标定的Demo。

3.3 角点提取Demo

```

srcPath: ./camera0_01_000.yuv
dstPath: ./camera0_01_000.jpg
boardWidth: 15 //number of
chessboard corners in horizontal direction
boardHeight: 8 //number of
chessboard corners in vertical direction
squareSize: 30 //size of
chessboard(mm)
imageWidth: 1920 //image width
imageHeight: 1080 //image height
imageStride: 1920 //image stride
imageFormat: 2 //0-gray, 1-rgb, 2-
yuv

```

以上为角点提取的配置参数。

```

//角点提取读取配置文件
int readFindPointsConfig(std::string readPath, std::vector<std::string>&
stringData, std::vector<int32_t>& int32Data)
{
    std::string stringLine;
    std::ifstream ifs(readPath, std::ifstream::in);
    if (ifs.fail())
    {
        std::cout << "read calib product config fail!" << std::endl;
    }
    int32_t lineNum = 0;
    int32_t valueData = 0;
    while (getline(ifs, stringLine))
    {
        std::stringstream ss(stringLine);
        std::string stringUnit;
        std::getline(ss, stringUnit, ' ');
        std::getline(ss, stringUnit, ' ');
        if (lineNum < 2)
        {
            stringData.push_back(stringUnit);
        }
        else
        {
            valueData = atoi(stringUnit.c_str());
            int32Data.push_back(valueData);
        }
        lineNum++;
    }
    ifs.close();
}

```

```

    return 0;
}

```

readFindPointsConfig函数用于读取角点提取的配置文件。

```

//角点提取Demo
int findPointsDemo(std::string configPath)
{
    std::vector<std::string> stringData;
    std::vector<int32_t> int32Data;
    readFindPointsConfig(configPath, stringData, int32Data);
    //获取版本信息
    char avscalibVersion[128];
    rkAVS_getCalibVersion(avscalibVersion);

    //初始化参数
    AVS_CALIB_FIND_POINTS_S stInputParams;
    //棋盘格规格
    stInputParams.stBoardParams.u32BoardWidth = int32Data[0];
    stInputParams.stBoardParams.u32BoardHeight = int32Data[1];
    stInputParams.stBoardParams.u32SquareSize = int32Data[2];
    //图像读取路径
    std::string srcImagePath = stringData[0];
    std::string dstImagePath = stringData[1];
    //图像规格
    uint8_t* pu8SrcImage;
    cv::Mat imageMat;
    int32_t dataSize;
    stInputParams.stImageData.stImageParams.u32ImageWidth = int32Data[3];
    stInputParams.stImageData.stImageParams.u32ImageHeight = int32Data[4];
    stInputParams.stImageData.stImageParams.u32ImageStride = int32Data[5];

    switch (int32Data[6])
    {
    case 0:
        stInputParams.stImageData.stImageParams.eImageFormat =
        RK_AVS_IMAGE_FORMAT_TYPE_GRAY;
        imageMat = cv::imread(srcImagePath, 0);
        pu8SrcImage = imageMat.ptr<uint8_t>(0);
        break;
    case 1:
        stInputParams.stImageData.stImageParams.eImageFormat =
        RK_AVS_IMAGE_FORMAT_TYPE_RGB;
        imageMat = cv::imread(srcImagePath, 1);
        pu8SrcImage = imageMat.ptr<uint8_t>(0);
        break;
    case 2:
        stInputParams.stImageData.stImageParams.eImageFormat =
        RK_AVS_IMAGE_FORMAT_TYPE_YUVNV12;
        dataSize = stInputParams.stImageData.stImageParams.u32ImageStride *
        stInputParams.stImageData.stImageParams.u32ImageHeight * 3 / 2;
        imageMat = cv::Mat(stInputParams.stImageData.stImageParams.u32ImageHeight
        * 3 / 2, stInputParams.stImageData.stImageParams.u32ImageStride, CV_8UC1);
        pu8SrcImage = imageMat.ptr<uint8_t>(0);
        readBinParams(srcImagePath, (char*)pu8SrcImage, dataSize);
        break;
    default:

```

```

        std::cout << "RK_AVS_CALIB: ImageFormat error!" << std::endl;
        return 1;
    }
    //输入输出路径
    stInputParams.pcPointsImagePath = dstImagePath.c_str();
    //读取图像
    stInputParams.stImageData.pu8ImageData = pu8SrcImage;

    //计时函数
#ifdef _WIN32
    clock_t startTime, endTime, timeAll;
    startTime = clock();
#else
    struct timeval startTime, endTime; // 秒+微妙
    time_t timeAll, timeAll_us;
    gettimeofday(&startTime, NULL);
#endif
    /*****角点提取*****/
    float stVerifyRms;
    int32_t status = rkAVS_findPoints(&stInputParams);
    if (status)
    {
        printf("rkAVS_calibFindPoints fail!\n");
    }
#ifdef _WIN32
    endTime = clock();
    timeAll = endTime - startTime;
    printf("rkAVS_calibFindPoints: %d ms\n", timeAll);
#else
    gettimeofday(&endTime, NULL);
    timeAll_us = 1000000 * (endTime.tv_sec - startTime.tv_sec) + (endTime.tv_usec
- startTime.tv_usec);
    printf("rkAVS_verifyProductStereo: %d us\n", timeAll_us);
#endif
    return 0;
}

```

findPointsDemo为角点提取的demo,角点提取的结果会按照指定路径保存。

4. 标定失败及精度差的原因

4.1 拍图不符合要求

下面列出了错误拍摄的标定板例子。

4.1.1 标定板反光

图4-1 反光



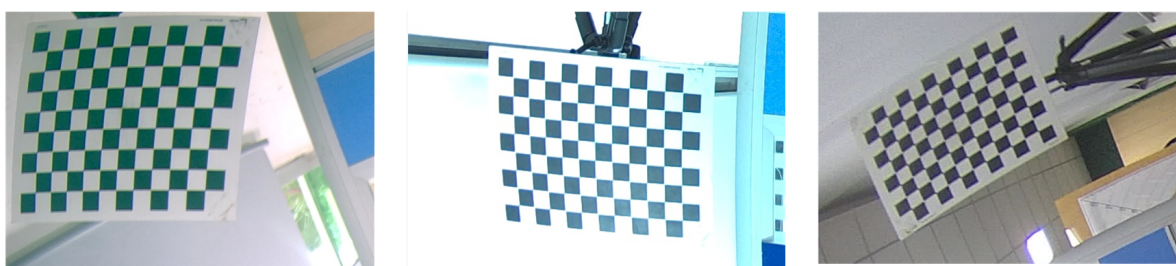
4.1.2 标定板未拍全

图4-2 棋盘格缺失



4.1.3 图像模糊

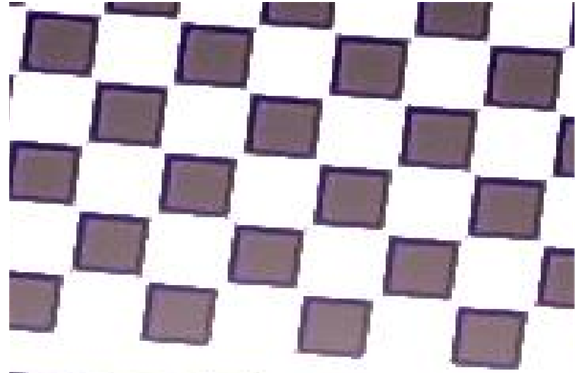
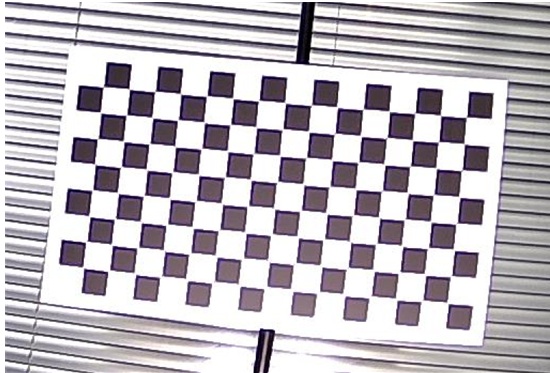
图4-3 棋盘格模糊



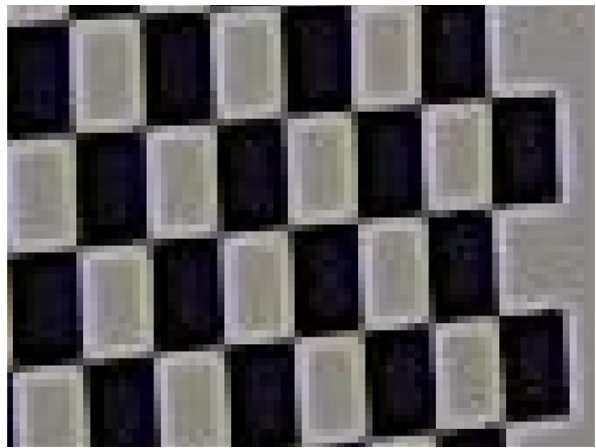
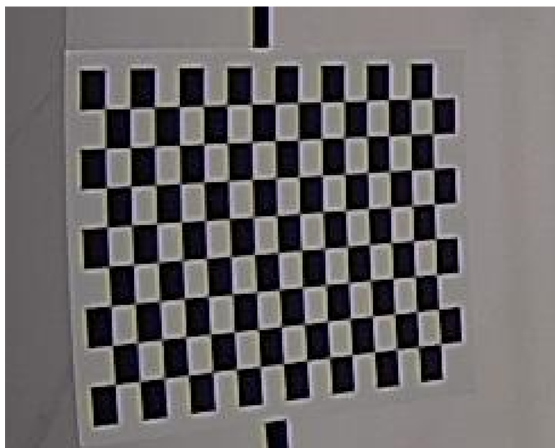
4.1.4 棋盘格不平整

张贴棋盘格不平整，存在鼓包等现象。

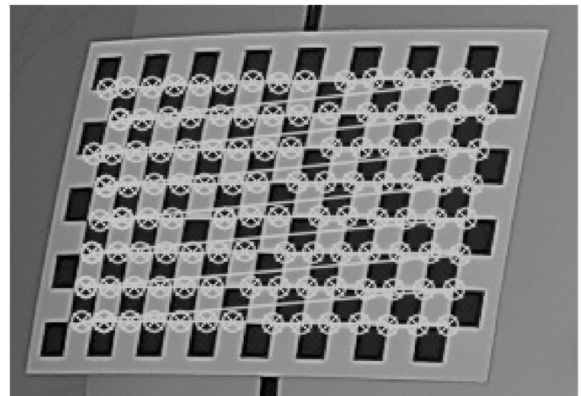
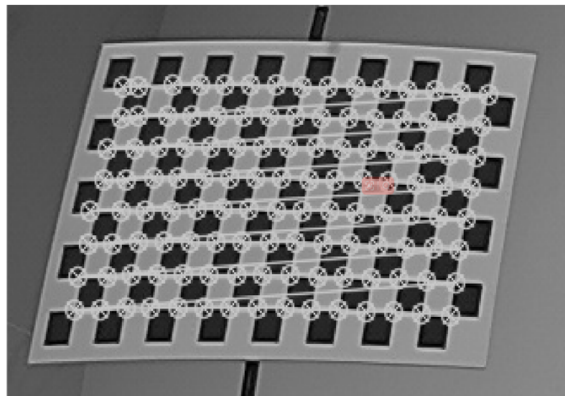
4.1.5 棋盘格过曝



4.1.6 光照不足



4.1.7 角点提取出错



角点提取出错的情况需要分析出错的原因，例如：标定板过曝、拍摄光照较差、棋盘格尺寸过小。

4.2 参数配置错误

- 棋盘格角点个数配置出错

以图2-1为例子，该棋盘格横向有16个格子，纵向有9个格子，每个格子30mm，横向有15个内点，纵向有8个内点，配置棋盘格参数时，应按以下配置：

```
//棋盘格规格
stInputParams.stBoardParams.u32BoardWidth = 15;
stInputParams.stBoardParams.u32BoardHeight = 8;
stInputParams.stBoardParams.u32SquareSize = 30;
```

- 图像尺寸配置出错

输入图像的宽、高、步长应配置正确。

5. 板端多目拼接标定流程

以下展示的是用命令行的方式采集标定图片，对于rv1126/rv1106/rk3588/rk3576是通用的，开发者也可以基于此原理来进行上位机的开发。注意，rv1126/rv1106还有单独的pc gui工具可以使用，操作上会更便捷。

5.1 准备工作

- 1) 准备好一块rv1126/rv1106/rk3588/rk3576开发板，确定开发板能够同步输出多目摄像头的YUV数据流，并且采用非融合拼接成一路数据流，经过编码推流出来；
- 2) 准备好网线以及数据线；

5.2 PC端访问开发板的方式

- 1) 普通adb: 将数据线与开发板连接，打开设备管理器界面确认PC端是否识别到Andorid设备，如已识别，则打开cmd，敲adb shell进入板端命令行进行操作，如未识别，则需使用瑞芯微驱动助手安装驱动；
- 2) 网络adb: 当数据线不够长的情况下，可以使用网线连接并访问开发板。步骤如下：
 - (a) 先使用普通adb的方式访问开发板；
 - (b) 在板端命令行输入以下命令，配置开发板的ip地址；

```
#ifconfig eth0 需要配置的ip地址 netmask 255.255.255.0
ifconfig eth0 192.168.1.30 netmask 255.255.255.0
```

- (c) 将PC的ip地址配到和上一步中开发板ip地址的同一网段，如将PC的ip地址配成 192.168.1.25
- (d) 打开 PC端命令行，输入如下指令连接开发板，剩下的使用方式与普通adb一样，敲adb shell进入板端即可；

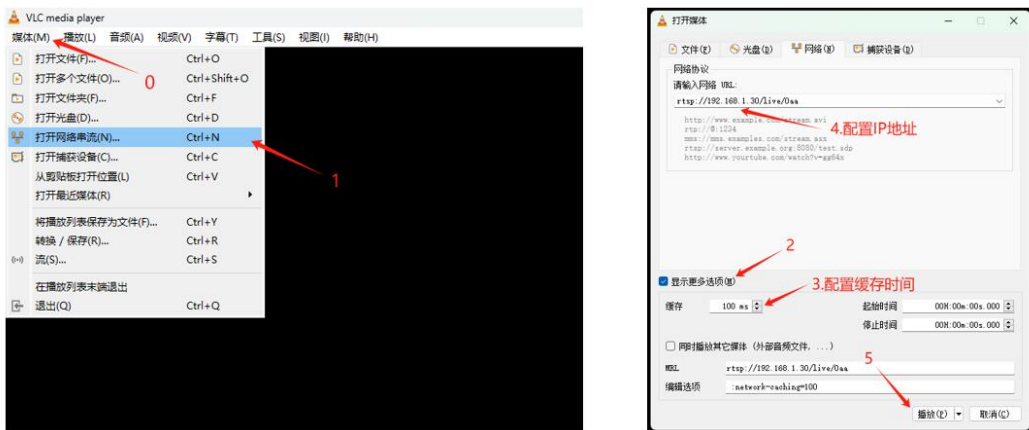
```
adb connect 192.168.1.30:5555
```

如果需要断开连接，则输入：


```
adb disconnect 192.168.1.30
```

5.3 视频预览

- 1) 在PC端安装VLC media player: <https://www.videolan.org/vlc/>;
- 2) 将开发板与PC使用网线连接;
- 3) 在开发板命令行敲ifconfig, 查看并记录开发板的ip地址;
- 4) 在开发板命令行敲rkipc &, 启动板端视频采集功能;
- 5) 在PC端打开VLC media player, 依次点击‘媒体’、‘打开网络串流’、‘显示更多选项’、‘将缓存时间调短, 一般100ms即可, 如果出现花屏, 则调长缓存时间’、‘输入网络URL’、'rtsp://开发板ip地址/live/0'、点击播放, 即可在PC端预览视频。



5.4 图像采集

5.4.1 多目拼接标定数据组别

假设目前需要标定的机器为多目结构, 则需要对每个相机进行内参采图, 以及每对相邻相机进行外参采图。内参数据组别命名规则为: camera + 相机编号 + _internal, 外参数据组别命名规则为: camera + 左相机编号 + 右相机编号 + _external。先在PC端建立好以数据组别为文件名的文件夹, 并放置在同一路径下。

5.4.2 采集标定数据

- 1) 确认已经按顺序执行完上述流程, 板端和PC正常连接, 且PC端能正常预览到板端采集的视频图像;
- 2) 检查板端/data/目录下是否有 'avs_record*' 开头的文件夹, 有的话就都删除:

```
rm -r /data/avs_record*
```

- 3) 确认当前采图组别, 将相机和标定板调整到合适位置, 进行标定图像采集, 采图的注意事项参考2.2和2.3节;
- 4) 采图过程中, 通过视频预览确认标定板位置和标定图像质量后, 板端敲命令抓图:

```
dumpsys avs record 0 1 2 /data/
```

同时保存一张AVS输入和输出文件到'/data/avs_record*'目录；

5) 重复步骤4，直至采集完当前组别所有标定图。先在板端打包'avs_record*'的文件夹：

```
cd /data
tar -cf avs_record_all.tar avs_record*
```

然后在pc上进入命令行界面，把数据拉到当前标定数据组别的文件夹中：

```
cd 标定数据存放文件夹
adb pull /data/avs_record_all.tar 标定组别路径
```

例如，若采集的是相机0的内参图, 则命令为:

```
cd D:/CalibData
adb pull /data/avs_record_all.tar ./camera0_internal
```

6) 在当前数据组别路径下，解压avs_record_all.tar；

7) 重复执行2~6步骤，直到采集完所有数据组别的标定数据。

5.4.3 使用python脚本重命名以及移动标定数据

```
# -*- coding: utf-8 -*-
import re
import shutil
import os
import sys

def processModeInternal(basedir, camera_id, output_dir):
    avs_list = os.listdir(basedir)
    pic_id = 0
    for name in avs_list:
        dir_child = os.path.join(basedir, name)
        if not os.path.isfile(dir_child):
            file_list = os.listdir(dir_child)
            for filename in file_list:
                source_file = os.path.join(dir_child, filename)
                if os.path.isfile(source_file):
                    matchObj = re.match(r'pipe_input_0_(\d*)_', filename)
                    if matchObj:
                        tmp_camera_id = matchObj.group(1)
                        if tmp_camera_id == str(camera_id):
                            if (pic_id >= 100):
                                newFilePath = os.path.join(output_dir, 'camera' +
                                str(camera_id) + '_' + str(pic_id) + '.yuv')
                                elif (pic_id >= 10):
                                    newFilePath = os.path.join(output_dir, 'camera' +
                                    str(camera_id) + '_0' + str(pic_id) + '.yuv')
                                else:
```

```

        newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id) + '_00' + str(pic_id) + '.yuv')
        shutil.copyfile(source_file, newFilePath)
        pic_id += 1

def processModeExternal(basedir, camera_id_left, camera_id_right, output_dir):
    avs_list = os.listdir(basedir)
    multi_char = '_' + str(camera_id_left) + str(camera_id_right)
    pic_id_left = 0
    pic_id_right = 0
    for name in avs_list:
        dir_child = os.path.join(basedir, name)
        if not os.path.isfile(dir_child):
            file_list = os.listdir(dir_child)
            for filename in file_list:
                source_file = os.path.join(dir_child, filename)
                if os.path.isfile(source_file):
                    matchObj = re.match(r'pipe_input_0_(\d*)_ ', filename)
                    if matchObj:
                        tmp_camera_id = matchObj.group(1)
                        if tmp_camera_id == str(camera_id_left):
                            if (pic_id_left >= 100):
                                newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_left) + multi_char + '_' + str(pic_id_left) + '.yuv')
                                elif (pic_id_left >= 10):
                                    newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_left) + multi_char + '_0' + str(pic_id_left) + '.yuv')
                                else:
                                    newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_left) + multi_char + '_00' + str(pic_id_left) + '.yuv')
                                    shutil.copyfile(source_file, newFilePath)
                                    pic_id_left += 1
                                if tmp_camera_id == str(camera_id_right):
                                    if (pic_id_right >= 100):
                                        newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_right) + multi_char + '_' + str(pic_id_right) + '.yuv')
                                        elif (pic_id_right >= 10):
                                            newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_right) + multi_char + '_0' + str(pic_id_right) + '.yuv')
                                        else:
                                            newFilePath = os.path.join(output_dir, 'camera' +
str(camera_id_right) + multi_char + '_00' + str(pic_id_right) + '.yuv')
                                            shutil.copyfile(source_file, newFilePath)
                                            pic_id_right += 1
                                if pic_id_right != pic_id_left:
                                    print("ERROR:PIC LOST.")

#板端标定数据存放的文件夹
basedir = 'D://CalibData//'
#相机个数
camera_num = 4
#相机是否成环
is_ring = True
#输出标定数据的文件夹
output_dir = 'D://CalibData//data//'

```

```
os.makedirs(output_dir, exist_ok=True)

#处理内参数据
for id in range(0, camera_num):
    picDir = basedir + 'camera' + str(id) + '_internal'
    processModeInternal(picDir, id, output_dir)

#处理外参数据
for id in range(0, camera_num - 1):
    picDir = basedir + 'camera' + str(id) + str(id + 1) + '_external'
    processModeExternal(picDir, id, id + 1, output_dir)

#如果相机成环,则需要处理编号0相机和最后一个相机的外参数据
if is_ring:
    picDir = basedir + 'camera' + str(camera_num - 1) + str(0) + '_external'
    processModeExternal(picDir, camera_num - 1, 0, output_dir)
```

注意, 如有需要, 可以修改此Python脚本中预设的文件夹路径。

5.4.4 内外参标定

经过上述所有流程, 板端采集的图像数据将被上述python脚本处理成符合规范的标定数据, 参考[第3.2章节](#)中的内容, 直接通过AVS标定工具进行标定, 获取标定后的文件即可。

5.5 其它注意事项

- 1) 对于每一数据组别, 在采集标定数据的时候, 要确保rkipc进程正常运行, 开发板不能断电重启;
- 2) 每采集完一个组别的标定数据, 就需要将该组数据从板端pull到PC端的指定文件夹中。